Highly Scalable and Reliable Pinterest Homepage with

Masonry Layout System Design

Highly Scalable and Reliable Pinterest Homepage with
Masonry Layout System Design

Generated on 2025-08-10 02:56 UTC

One Pager Summary

Goal: Deliver a fast, personalized Pinterest style homepage with an adaptive masonry layout that streams visual content quickly and supports infinite scrolling at global scale.

Users: End users browsing pins, creators publishing content, advertisers, and administrators. Multi tenant support for brands or regional catalogs.

Core Value: Pixel stable masonry grid with low cumulative layout shift, rapid image paint, and high relevance ranking. Low integration friction for web and mobile with shared SDKs.

North Star Metrics: p95 time to first contentful paint under 1.2 seconds on median devices, p95 feed fetch under 150 milliseconds from edge, p95 publish to visibility under 2 seconds, availability 99.95 percent or better.

Scope: Client rendering and layout, image delivery, feed assembly and ranking, interactions, moderation, ads insertion, observability, and operational excellence.

Functional Requirements

Homepage renders a masonry grid of pins that adapts to viewport and density settings. Infinite scroll with append and smooth column balancing. Interactive elements include save to board, like, comment, share, and follow. Hover and long press quick actions. Rich pin cards with title, description, creator, and attribution. Search and discovery through query and related pins. Optional ads with frequency and placement rules. Accessibility including keyboard navigation and screen readers. Localization and right to left support. Offline hints such as prefetch and optimistic interactions. User settings for content preferences and safe search. Admin and moderation tools for content takedown, quality labels, and brand safety. Analytics for engagement, latency, and errors.

Non

Availability 99.95 percent platform wide, 99.99 percent for read heavy endpoints. Latency p95 feed read under 150 milliseconds from edge cache, image CDN p95 under 100 milliseconds for first byte. Consistency model is read your writes for author on own content, eventual global convergence under 2 to 5 seconds. Durability six nines for operational data and eleven nines for event logs and media references. Scalability supports 2 million feed reads per second globally and 200 thousand writes per second across interactions. Privacy and compliance aligned with SOC 2 type 2, GDPR, and CCPA. Cost efficiency through edge caching, image transformation at the CDN, and tiered storage.

Architecture Overview

Client: React or Next style app with server side rendering or static generation for shell, streaming server components for critical path, and client hydration with prioritized routes. Headless masonry component packaged as a design system primitive. TypeScript SDK shared across web and mobile web. State managed with React Query or equivalent for caching, background revalidation, and prefetch. Feature flags for experiments.

Edge: CDN and edge compute for HTML and JSON caching, image resizing and format negotiation, token verification, and bot screening. Edge KV for short lived feed segments and experiment assignments.

Gateway: API Gateway with WAF and DDoS protection. REST and GraphQL interfaces. Request routing by user affinity and region.

Core Services

Identity and Session Service with OAuth and OIDC.

Content Service for pins, media, metadata, and attribution.

Feed Orchestrator for candidate generation and assembly.

Ranking Service for personalization and diversity constraints.

Social Graph Service for follows, boards, and interests.
Timeline Store for materialized home feed segments.
Ads Service for supply, targeting, and pacing.
Media Service for upload, transcode, dedupe, and CDN distribution.
Interaction Service for saves, likes, comments, and shares.
Moderation Service for policy checks and quality labels.
Search Service for query and related pins.
Analytics and ETL for events and model training.
Masonry Component Design
Props include column count or breakpoint map, column gap, row gap, minimum card width, aspect mode, estimated image height, overscan factor, virtualization window size, layout strategy, on render and on visible callbacks, and placeholders. Accessibility props expose list semantics and focus management.
Layout approaches
Client measured layout with resize observer to compute column count and container width. Place items using shortest column first with a min heap keyed by current column height. Server assisted layout when dimensions are known using predicted heights sent with feed. Hybrid approach starts with predicted heights to avoid reflow then corrects using actual image metadata on load. Ensure zero layout shift by reserving space based on width and height ratio known from media variants.
Virtualization
Window based virtualization with sentinel elements and intersection observer. Prepend and append windows with equal column heights tracking. Smooth append by batch commit and animation. Maintain stable keys to

avoid reflow of rendered cells.
Image loading
Use CDN variants with width breakpoints and modern formats. Lazy load off screen images with fetch priority for above the fold. Progressive decode using preview thumbnails that blur into full images. Preconnect to CDN and use priority hints.
Data Flow
Page load
Server renders shell and first screen feed segment with predicted heights. Edge caches HTML and first segment keyed by user and experiment cohort. Client hydrates, attaches masonry, and starts prefetch of next segments.
Scroll
Intersection observer triggers fetch of next segment. Feed Reader merges materialized timeline with on demand expansions for heavy hitters. Masonry computes placement using column heap and inserts a batch. Client emits visibility events for analytics and ranking features.
Publish to visibility
Creator posts to Content Service. Event is appended to log. Feed Orchestrator computes candidate sets using Social Graph and interest signals. For normal accounts fan out on write precomputes segments. For heavy hitters defer to fan out on read with pointers. Timeline Store updates and notifies edge cache for soft refresh.
Image delivery
Client requests best fit rendition. CDN resizes and serves with low latency. Client swaps preview to full image on decode.
Technology Stack
Client: TypeScript, React or Next, headless component library, CSS grid for fallbacks, Resize Observer, Intersection Observer, Web Workers for layout off main thread when needed.
Edge: CDN with image transformation and edge compute.
Gateway: Envoy or API Gateway and WAF.
Messaging: Kafka or Kinesis partitioned by user id and pin id.
Operational Data: Postgres or DynamoDB for pins and metadata with read replicas. Row level security for multi tenancy.
Timeline Store: Redis Cluster for hot segments and Cassandra or DynamoDB for longer history.
Search: OpenSearch or Elastic managed service.

Feature Store and Ranking: Online features in Redis or DynamoDB, offline features in data lake. Model serving on a managed platform.
Media: Object storage for originals and variants, CDN for delivery.

Observability: OpenTelemetry, Prometheus, Grafana, centralized logs.
CI CD and IaC: Git based pipelines, Terraform, progressive delivery with canaries and feature flags.
Data Model Sketch
User: id, handle, profile, locale, preferences, createdAt, metrics.
Board: id, userId, title, description, visibility, createdAt, metrics.
Pin: id, authorId, boardId optional, title, description, tags, media variants with width and height and url and type, attribution, quality labels, createdAt, editedAt, version.
FeedItem: ownerUserId, pinId, score, createdAt, source, shardId.
Interaction: id, userId, pinId, type save or like or comment, ts, meta.
AdPlacement: id, slot, targeting, pacing, creativeRef, ts.
ModerationCase: id, entityId, type, status, actions, reviewerId, ts.
Scaling and

Storage Strategy

Hybrid fan out to balance cost and tail latency. Pre materialize home feed segments for most users while deferring celebrity content expansion until read time. Shard timeline by user with segments stored as ordered lists. Redis holds hot segments and recent interactions. Cassandra or DynamoDB persists long tail. Maintain per user per device breakpoints for predicted heights to reduce reflow.
Image storage uses object storage with multiple renditions and perceptual dedupe. CDN edge caches hot images and JSON segments. Avoid origin reads by keeping first segments at edge KV.
Counters for likes and saves use atomic increments with periodic reconciliation. Search index updated near real time from event log. Cold content migrates to cheaper tiers with on demand rehydration.
Latency Budgets and Throughput
Time to first contentful paint p95 goal 1.2 seconds on median devices with fast network. Above the fold JSON segment from edge in under 80 milliseconds. Image first byte from CDN under 100 milliseconds with preconnect. Publish to visibility p95 under 2 seconds. Interactions p95 under 120 milliseconds with optimistic updates. Throughput example 2 million feed reads per second, 20 million image requests per second at peak, 5 million interactions per minute during events.
SLA and SLOs
Platform availability 99.95 percent monthly. Edge served feed reads 99.99 percent. Error budget 0.5 percent. RPO zero for event logs and media references. RTO per region under 15 minutes with warm standby or active active topology.
Ads and Monetization

Slot level contracts define max density, frequency caps, and brand safety. Ads are inserted by the Feed Orchestrator using pacing and performance feedback. Ensure click protection and viewability measurement. Keep user privacy by separating ad decisioning data and applying purpose based access controls.
Moderation and Safety
Pre publish checks for policy with lightweight classifiers. Post publish monitoring for adult and violence with quality labels and regional rules. User tools to report, mute, or block. Shadow actions for suspected abuse. Legal hold exports and immutable audit logs.
Security Privacy and Compliance
PII minimization and pseudonymous identifiers by default. Encryption in transit and at rest with KMS and automatic rotation. Secret management with dedicated KMS and short lived tokens. Data residency per tenant as needed. GDPR and CCPA tooling for subject requests and deletion. SOC 2 controls for change management, access, and monitoring. Vendor risk and annual pen tests.
Multi Tenancy
Logical isolation by tenant id with row level security and per tenant keys. Per tenant domains, themes, and ranking rules. Rate limits and quotas per tenant. Enterprise options for dedicated VPC and regional isolation. Separate billing and cost attribution by tenant.

APIs
Public Feed API
GET v1 feed home
Parameters userId, cohort, cursor, limit, locale, device class
Response array of feed items with pin and predictedCardSize and nextCursor
Interaction API
POST v1 pins pinId save
POST v1 pins pinId like
POST v1 pins pinId comment
Search API
GET v1 search pins q cursor limit filters
Admin APIs
POST v1 pins, PATCH v1 pins pinId, POST v1 moderation actions, GET v1 analytics. All endpoints versioned and idempotent where applicable.

CI CD and Deployment
Trunk based development with automated tests and security scans. Infrastructure as code with Terraform and environment workspaces. Canary releases for feed and ranking with shadow reads and metric based rollback. Feature flags for masonry algorithm variants and image pipeline changes. Blue green for media and search clusters with health gates and warmup.

Observability and Operations

Distributed tracing across edge, gateway, services, and data stores. Metrics include time to first contentful paint, p95 and p99 latencies, cache hit rate, image decode time, layout work time, CLS, LCP, consumer lag, and error budgets. Logs with redaction and sampling. Alerting on SLO burn, image errors, cache miss spikes, shard hotspots, and moderation backlog. Runbooks and dashboards per region and tenant. RUM for client side performance and correlation to backend traces.
Capacity Planning and Cost
Reference per region sizing
Edge cache covers first two segments and top image variants to keep origin reads under five percent. Redis cluster with twelve shards for timeline hot data. Kafka or Kinesis three to five brokers with three day retention. Cassandra or DynamoDB sized for write rate with auto scaling and global tables where needed. Search cluster with three data nodes and two masters. Media egress optimized via CDN. Estimated monthly cost in low to mid six figures depending on regions, traffic mix, and media quality settings. Cost levers include cache ttl, image variant counts, and pre materialized segment depth.

Testing Strategy and Failure Drills

Unit tests for layout, virtualization, and image helpers. Property tests for column balancing and stability. Visual regression tests on a matrix of breakpoints and densities. Contract tests for APIs. Load tests for scroll and image bursts. RUM based experiments for layout variants. Security testing with SAST, DAST, dependency scans, and secret scanning.
Failure drills include cache eviction storms, image CDN degradation, shard loss in timeline store, ranking timeouts, and regional failover. Validate bounded latency increases, zero data loss, and automatic recovery. Practice runbooks and capture MTTR.

Phased Build Plan

MVP four to six weeks
Server rendered shell, headless masonry with predicted heights, infinite scroll, edge cached first segment, save and like interactions, single region with multi AZ, metrics and dashboards.
Phase 2 two months
Hybrid fan out, ranking service, ads insertion, advanced moderation, media pipeline with preview to full swap, canary deploys, chaos tests, and localization.

Phase 3 three to six months

Active active multi region, semantic discovery, data residency, enterprise tenant isolation, smarter layout with off main thread computation, and marketplace integrations.

Key Design Choices Trade Offs and Risks

Masonry layout stability versus flexibility

Predicted heights reduce layout shift but may be wrong. Mitigate with quick correction after image metadata load and gentle reflow animations.

Hybrid fan out introduces complexity

It keeps costs and tail latency in check but requires strong observability and backfill. Mitigate with clear thresholds and hot account detection.

Edge caching staleness risk

Low ttl and push invalidation for hot segments reduce staleness while keeping latency low.

Media costs

Image transformation and egress can dominate costs. Mitigate by limiting variant counts, using modern formats, and aggressive CDN caching.